Marketplace Implementation Plan

Overview

Complete the Buyer Product Browse page (`/buyer/products`) with full search, sorting, and product display functionality.

Target Page: `/buyer/products` (ProductListComponent)

Current State

Already Implemented:

- Product grid with responsive layout
- Filter by Grade (Premium, Grade I, II, III)
- Filter by Price Range (min/max)
- Filter by Origin (text search)
- Pagination (12 items per page)
- Product cards with image, name, price, rating, badges

Missing:

- Search functionality (product name/description)
- Sorting options
- Accurate total count from backend

Features to Implement

Feature	Description
Search	Full-text search on product name and description
Sort: Price Low→High	Order by minimum variant price ascending
Sort: Price High→Low	Order by minimum variant price descending
Sort: Best Sellers	Order by number of orders (most sold first)

Step 1: Backend - Update Product Service

File: `backend/src/services/product.service.ts`

1.1 Add Search Parameter

Add `search` parameter to filter products by name or description:

```
// In listProducts method parameters
search?: string;

// In WHERE clause
if (search) {
  conditions.push(`(sp.product_name LIKE ? OR sp.description LIKE ?)`);
  params.push(`%${search}%`, `%${search}%`);
}
```

1.2 Add Sort Parameter

Add `sort` parameter to control ordering:

```
// In listProducts method parameters
sort?: 'price_asc' | 'price_desc' | 'best_sellers' | 'newest';

// Replace hardcoded ORDER BY with dynamic sorting
let orderBy = 'sp.created_at DESC'; // default

switch (sort) {
  case 'price_asc':
    orderBy = 'min_price ASC';
    break;
  case 'price_desc':
    orderBy = 'min_price DESC';
    break;
  case 'best_sellers':
    orderBy = 'order_count DESC, sp.created_at DESC';
    break;
  case 'newest':
    orderBy = 'sp.created_at DESC';
    break;
}
```

1.3 Add Order Count for Best Sellers

Modify the SQL query to include order count:

```
SELECT sp.*,
  s.business_name AS seller_name,
  MIN(pv.price) AS min_price,
  MAX(pv.price) AS max_price,
  SUM(pv.stock_quantity) AS total_stock,
  ROUND(AVG(r.rating), 1) AS average_rating,
  COUNT(DISTINCT r.id) AS review_count,
  COUNT(DISTINCT oi.id) AS order_count
FROM saffron_products sp
```

```
JOIN sellers s ON sp.seller_id = s.id
LEFT JOIN product_variants pv ON sp.id = pv.product_id
LEFT JOIN reviews r ON sp.id = r.product_id
LEFT JOIN order_items oi ON pv.id = oi.variant_id
WHERE sp.status = 'active'
  [AND search conditions]
  [AND filter conditions]
GROUP BY sp.id
ORDER BY [dynamic order]
LIMIT ? OFFSET ?
```

1.4 Return Total Count

Ensure the response includes accurate total count for pagination:

```
// Run count query separately
const countQuery = `SELECT COUNT(DISTINCT sp.id) as total FROM saffron_products sp WHERE ...

return {
  products: results,
  total: countResult.total,
  page: page,
  limit: limit
};
```

Step 2: Backend - Update Validators

File: `backend/src/utils/marketplace-validators.ts`

Add validation for new query parameters:

```
// In productQuerySchema
search: Joi.string().max(100).allow(''),
sort: Joi.string().valid('price_asc', 'price_desc', 'best_sellers', 'newest')
```

Step 3: Backend - Update Routes

File: `backend/src/routes/product.routes.ts`

Extract and pass new query parameters:

```
router.get('/', async (req, res) => {
  const {
    grade,
    min_price,
    max_price,
```

```
    origin,
    search,     // NEW
    sort,       // NEW
    page,
    limit
  } = req.query;

  const result = await productService.listProducts({
    grade,
    minPrice: min_price ? Number(min_price) : undefined,
    maxPrice: max_price ? Number(max_price) : undefined,
    origin,
    search,     // NEW
    sort,       // NEW
    page: page ? Number(page) : 1,
    limit: limit ? Number(limit) : 12
  });

  res.json({ success: true, data: result });
});
```

Step 4: Frontend - Update Product Service

File: `frontend/src/app/core/services/product.service.ts`

4.1 Update getProducts Method

Add search and sort parameters:

```
getProducts(filters?: {
  grade?: string;
  minPrice?: number;
  maxPrice?: number;
  origin?: string;
  search?: string;     // NEW
  sort?: string;       // NEW
  page?: number;
  limit?: number;
}): Observable<ApiResponse<{ products: ProductResponse[], total: number, page: number, limit
  let params = new HttpParams();

  if (filters?.search) {
    params = params.set('search', filters.search);
  }
  if (filters?.sort) {
```

```
    params = params.set('sort', filters.sort);
  }
  // ... existing params

  return this.http.get<ApiResponse<...>>(this.apiUrl, { params });
}
```

Step 5: Frontend - Update ProductListComponent TypeScript

File: `frontend/src/app/features/buyer/products/product-list/product-list.component.ts`

5.1 Add New Properties

```
// Search
searchQuery: string = '';

// Sort
sortOptions = [
  { value: '', label: 'Default' },
  { value: 'price_asc', label: 'Price: Low to High' },
  { value: 'price_desc', label: 'Price: High to Low' },
  { value: 'best_sellers', label: 'Best Sellers' }
];
selectedSort: string = '';
```

5.2 Add New Methods

```
onSearch(): void {
  this.filters.page = 1;
  this.loadProducts();
}

onSortChange(): void {
  this.filters.page = 1;
  this.loadProducts();
}
```

5.3 Update loadProducts Method

```
loadProducts(): void {
  this.isLoading = true;
  this.error = null;

  const filters = {
```

```
    page: this.filters.page,
    limit: this.filters.limit,
    search: this.searchQuery.trim() || undefined,    // NEW
    sort: this.selectedSort || undefined,            // NEW
    grade: this.selectedGrade || undefined,
    minPrice: this.minPrice || undefined,
    maxPrice: this.maxPrice || undefined,
    origin: this.originSearch.trim() || undefined
  };

  this.productService.getProducts(filters)
    .pipe(takeUntil(this.destroy$), finalize(() => this.isLoading = false))
    .subscribe({
      next: (response) => {
        if (response.success && response.data) {
          this.products = response.data.products;
          this.totalProducts = response.data.total;  // Use actual total
        }
      },
      error: (error) => {
        this.error = 'Failed to load products. Please try again.';
      }
    });
}
```

5.4 Update hasActiveFilters Getter

```
get hasActiveFilters(): boolean {
  return !!(
    this.selectedGrade ||
    this.minPrice ||
    this.maxPrice ||
    this.originSearch.trim() ||
    this.searchQuery.trim()  // NEW
  );
}
```

5.5 Update clearFilters Method

```
clearFilters(): void {
  this.selectedGrade = '';
  this.minPrice = null;
  this.maxPrice = null;
  this.originSearch = '';
  this.searchQuery = '';     // NEW
  this.selectedSort = '';    // NEW (optional - keep sort on clear?)
```

```
  this.filters.page = 1;
  this.loadProducts();
}
```

Step 6: Frontend - Update Template

File: `frontend/src/app/features/buyer/products/product-list/product-list.component.html`

6.1 Update Header Structure

Replace the existing header with:

```
<header class="page-header">
  <div class="header-top">
    <div class="header-branding">
      <h1>Saffron Marketplace</h1>
      <p class="header-subtitle">Discover premium quality saffron from verified sellers</p>
    </div>
    <a routerLink="/buyer/cart" class="cart-link">
      <svg viewBox="0 0 24 24"><path d="M7 18c-1.1 0-1.99.9-1.99 2S5.9 22 7 22s2-.9 2-2-.9-2
      Cart
    </a>
  </div>

  <div class="header-controls">
    <!-- Search Bar -->
    <div class="search-bar">
      <svg class="search-icon" viewBox="0 0 24 24"><path d="M15.5 14h-.79l-.28-.27C15.41 12.
      <input
        type="text"
        placeholder="Search saffron products..."
        [(ngModel)]="searchQuery"
        (keyup.enter)="onSearch()"
      />
      <button class="search-btn" (click)="onSearch()">Search</button>
    </div>

    <!-- Sort Dropdown -->
    <div class="sort-control">
      <label for="sort-select">Sort:</label>
      <select id="sort-select" [(ngModel)]="selectedSort" (change)="onSortChange()">
        @for (option of sortOptions; track option.value) {
          <option [value]="option.value">{{ option.label }}</option>
        }
      </select>
```

```
    </div>

    <!-- Filter Toggle Button -->
    <button class="filter-toggle-btn" (click)="toggleFilters()">
      <svg viewBox="0 0 24 24"><path d="M10 18h4v-2h-4v2zM3 6v2h18V6H3zm3 7h12v-2H6v2z"/></s
      Filters
      @if (hasActiveFilters) {
        <span class="filter-badge"></span>
      }
    </button>
  </div>
</header>
```

6.2 Add Results Summary

Add above the products grid:

```
<div class="results-summary">
  @if (searchQuery.trim()) {
    <span>Results for "{{ searchQuery }}"</span>
  }
  <span class="product-count">{{ totalProducts }} product(s) found</span>
</div>
```

Step 7: Frontend - Update Styles

File: `frontend/src/app/features/buyer/products/product-list/product-list.component.css`

7.1 Header Controls Layout

```
.header-top {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  margin-bottom: 1.5rem;
}

.header-branding h1 {
  margin: 0;
  font-size: 1.75rem;
  color: #333;
}

.header-subtitle {
  margin: 0.25rem 0 0;
```

```
  color: #666;
  font-size: 0.875rem;
}

.header-controls {
  display: flex;
  align-items: center;
  gap: 1rem;
  flex-wrap: wrap;
}
```

7.2 Search Bar Styles

```
.search-bar {
  display: flex;
  align-items: center;
  flex: 1;
  min-width: 200px;
  max-width: 400px;
  background: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  overflow: hidden;
}

.search-bar .search-icon {
  width: 20px;
  height: 20px;
  margin-left: 12px;
  fill: #666;
}

.search-bar input {
  flex: 1;
  padding: 0.75rem;
  border: none;
  outline: none;
  font-size: 0.875rem;
}

.search-bar input::placeholder {
  color: #999;
}

.search-btn {
  padding: 0.75rem 1.25rem;
```

```
  background: #8b5a2b;
  color: white;
  border: none;
  font-weight: 500;
  cursor: pointer;
  transition: background 0.2s;
}

.search-btn:hover {
  background: #6d4522;
}
```

7.3 Sort Control Styles

```
.sort-control {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.sort-control label {
  font-size: 0.875rem;
  color: #666;
  white-space: nowrap;
}

.sort-control select {
  padding: 0.625rem 2rem 0.625rem 0.75rem;
  border: 1px solid #ddd;
  border-radius: 6px;
  background: white url("data:image/svg+xml,...") no-repeat right 0.5rem center;
  background-size: 16px;
  font-size: 0.875rem;
  cursor: pointer;
  appearance: none;
}

.sort-control select:focus {
  outline: none;
  border-color: #8b5a2b;
}
```

7.4 Results Summary Styles

```
.results-summary {
  display: flex;
```

```
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1rem;
  padding: 0.75rem 1rem;
  background: #f9f9f9;
  border-radius: 6px;
  font-size: 0.875rem;
  color: #666;
}

.product-count {
  font-weight: 500;
}
```

7.5 Responsive Styles

```
@media (max-width: 768px) {
  .header-controls {
    flex-direction: column;
    align-items: stretch;
  }

  .search-bar {
    max-width: 100%;
  }

  .sort-control {
    justify-content: space-between;
  }
}

@media (max-width: 480px) {
  .header-top {
    flex-direction: column;
    gap: 1rem;
  }

  .cart-link {
    align-self: flex-end;
  }
}
```

Step 8: Database - Add Order Count Index (Optional)

For better performance on "Best Sellers" sorting:

```
CREATE INDEX idx_order_items_variant ON order_items(variant_id);
```

Step 9: Testing

9.1 Search Tests

Test	Expected Result
Search "Persian"	Shows products with "Persian" in name or description
Search "Negin"	Shows Negin saffron products
Search "xyz123"	Shows empty state "No products found"
Clear search	Shows all products
Search + Grade filter	Combined filtering works

9.2 Sort Tests

Test	Expected Result
Sort "Price: Low to High"	Cheapest products first
Sort "Price: High to Low"	Most expensive products first
Sort "Best Sellers"	Products with most orders first
Change sort while filtered	Results re-sorted correctly
Sort persists on pagination	Page 2 maintains sort order

9.3 Pagination Tests

Test	Expected Result
Total count shows correctly	Matches actual filtered results
Navigate to page 2	Shows next 12 products
Filter reduces total	Count updates accurately

Files Summary

File	Action	Description
`backend/src/services/product.service.ts`	MODIFY	Add search, sort, order_count logic
`backend/src/utils/marketplace-validators.ts`	MODIFY	Add search, sort validation
`backend/src/routes/product.routes.ts`	MODIFY	Extract new query params
`frontend/src/app/core/services/product.service.ts`	MODIFY	Add search, sort params

File	Action	Description
`frontend/src/app/features/buyer/products/product-list/product-list.component.ts`	MODIFY	Add search/sort methods
`frontend/src/app/features/buyer/products/product-list/product-list.component.html`	MODIFY	Add search bar, sort dropdown
`frontend/src/app/features/buyer/products/product-list/product-list.component.css`	MODIFY	Add search/sort styles

Build & Test Commands

```
# Backend
cd backend
npm run build
npm run dev

# Frontend
cd frontend
npm run build
npm run start

# Test URLs
# http://localhost:4200/buyer/products
# http://localhost:4200/buyer/products?search=persian
# http://localhost:4200/buyer/products?sort=price_asc
# http://localhost:4200/buyer/products?grade=premium&sort=best_sellers
```

Estimated Effort

Task	Lines of Code	Time
Backend service changes	~50	30 min
Backend validators/routes	~20	15 min
Frontend service	~15	10 min
Frontend component TS	~40	20 min
Frontend template	~50	20 min
Frontend styles	~100	30 min
Testing	-	30 min
Total	**~275**	**~2.5 hours**